

A Data-Driven Capacity-Reservation System for Cloud Cost Optimization under Intermittent Demand Surges

First Author*, Second Author*, and Third Author*

*Department of Scientific Computing, Faculty of Computer and Information Sciences

Affiliation Placeholder, City, Country

Email: {author1, author2, author3}@example.edu

Abstract—Enterprises increasingly run on public clouds, yet a large fraction of cloud spend is wasted because demand is a low, stationary *base* load punctuated by intermittent random *surges*, while pricing offers a discounted but committed *reserved* tariff alongside an expensive but elastic *on-demand* tariff. The operations-management literature characterizes the cost-optimal response as a capacity-reservation policy—a newsvendor sizing of *base* and *supplementary* reserved contracts with the residual met by on-demand spillover, governed by a two-threshold (s, S) rule—but stops at closed-form policies for stylized demand. We ask whether that policy can be turned into a deployable, data-driven system. We present Smart Cloud Optimizer, which (i) forecasts per-account cost and utilization with five time-series models selected by walk-forward cross-validation, (ii) sizes reserved capacity with a mixed-integer program whose right-hand sides are an empirical critical fractile (the 95th percentile of utilization inflated by a $1.3\times$ headroom), and (iii) applies eight rule-based actions for reserved/on-demand contract choice and service-specific waste. On a 365-day, multi-service workload, exponential smoothing attains 11.2% mean absolute percentage error at a 30-day horizon; the optimizer produces 19 recommendations worth \$590.40/month, a 27.0% reduction of the \$2189.48/month bill. We make explicit where the deployed system departs from the optimal policy. The evaluation workload is synthetic and modeled on public cloud traces, not a live account; we report this as a threat to validity.

Index Terms—cloud cost optimization, capacity reservation, newsvendor, reserved instances, time-series forecasting, mixed-integer programming, FinOps.

I. INTRODUCTION

Public-cloud adoption continues to accelerate, but cost governance has not kept pace: industry surveys consistently report that roughly a quarter to a third of cloud spend is wasted on idle or over-provisioned resources [1]. The root cause is a mismatch between how demand behaves and how capacity is priced. Demand for a typical workload is a *base* load—low, stationary, and predictable—punctuated by intermittent *surges* driven by promotions, launches, batch jobs, or seasonal events [2], [3]. Pricing, meanwhile, is two-sided: a *reserved* (committed) tariff that is cheap per unit but is paid whether used or not, and an *on-demand* tariff that is elastic with zero commitment but costs substantially more per unit. Choosing how much capacity to reserve versus leave to on-demand is therefore a classic trade-off between commitment risk and elasticity premium.

Chen, Lei, and Moinszadeh [4] formalize this trade-off as a capacity-reservation problem. A firm holds *base contracts* sized to the stationary base demand and *supplementary contracts* for surges; demand beyond reserved capacity spills to on-demand. They show that the cost-optimal capacities follow a *newsvendor critical fractile*, and that the optimal management of supplementary contracts (purchase, renewal, cancellation, expiration) is a two-threshold (s, S) policy. This is an elegant and complete *characterization*, but it presumes the demand distributions are known and deterministic in structure, and it produces a policy rather than an operational pipeline that an engineering team can run against a real account.

A parallel systems literature attacks the same waste empirically—forecasting utilization and reacting with autoscaling or instance acquisition [5]–[9]—but rarely connects its heuristics back to the cost-optimal reservation policy, and seldom reports end-to-end dollar savings with a reproducible methodology. The two bodies of work are complementary: one supplies the economic optimum, the other supplies the data and tooling.

This paper bridges the two. We present *Smart Cloud Optimizer*, a system that operationalizes the base/supplementary/on-demand reservation framework of [4] on real account telemetry. We adopt that paper’s vocabulary and sizing principle—base and supplementary reserved capacity, on-demand spillover, the newsvendor fractile, and the two-threshold (s, S) view—as the *lens* for a deployable pipeline built from forecasting, mixed-integer optimization, and rule-based actions. Our contributions are:

- 1) **A practical realization of the capacity-reservation policy.** We show that a deployed right-sizing rule—reserve to the 95th percentile of forecasted utilization with a $1.3\times$ headroom—is an *empirical critical fractile*, and we relate the implied service level to the cost-optimal newsvendor fractile (Section III).
- 2) **An end-to-end system** that forecasts cost/utilization (five models, walk-forward selection), sizes capacity with a mixed-integer program, and chooses reserved-vs-on-demand contracts and service-level waste removal with eight rules (Section IV).
- 3) **A reproducible numerical study** on a 365-day, multi-

service workload: exponential smoothing reaches 11.2% MAPE at a 30-day horizon, and the optimizer yields 19 recommendations worth \$590.40/month (27.0% of the bill). Every number is regenerated by a single script from the database (Section V).

- 4) **An honest account of the gap** between the deployed system and the optimal policy, and of the synthetic nature of the workload (Section VI).

The remainder follows the arc of [4]: Section II positions the work, Section III states the reservation model and newsvendor/(s, S) policy, Section IV describes the realizing system, Section V reports the numerical study, Section VI discusses threats to validity, and Section VII concludes.

II. RELATED WORK

We organize related work into the three streams of [4], extended with the forecasting stream we draw on.

(1) **Capacity reservation under random demand and surges.** The reservation/option-contract view of capacity has a long history in operations: backup and reservation contracts trade commitment for flexibility [10], [11], and planning for random demand surges combines reactive capacity with safety stock [12]. State-modulated (base-vs-surge) demand is modeled classically via Markov-modulated processes in inventory and queueing [13]–[15], and the optimal single-period commitment is the newsvendor critical fractile [16]. Our modeled reference [4] specializes this to cloud reserved/on-demand pricing with cancellation and a secondary marketplace; Furman and Diamant [17] study a complementary provider-side capacity problem under periodic time-varying demand. We borrow the *policy* from this stream and ask how to realize it.

(2) **Cloud cost optimization and instance acquisition.** On the cloud-user side, stochastic programming chooses reservation levels under demand and price uncertainty [5]; online and brokerage algorithms acquire reserved/on-demand instances over time [6], [18]; scheduling and instance switching cut rental cost [7]; and platform-side work studies overcommitment [19], slack capacity [20], bundled procurement [21], preemptible discounts [22], [23], and the reserved marketplace [24]. These works are largely algorithmic or analytical; we contribute a working pipeline that emits dollar-quantified recommendations.

(3) **Forecasting-driven provisioning.** Time-series forecasting underpins proactive provisioning [8], [9], [25]. We use established models—exponential smoothing/Holt–Winters [26], [27], automatic ARIMA [28], and Prophet [29] via statsmodels [30]—together with streaming anomaly detection in the spirit of the Numenta Anomaly Benchmark [31], [32]. To our knowledge, none of these works ties the forecast to the *newsvendor* reservation optimum and reports the realized account-level saving, which is our integrative aim.

III. RESERVATION MODEL AND POLICY

We restate the model of [4] in the notation of Table I, then derive the sizing principle our system approximates.

TABLE I
MODEL NOTATION (AFTER [4]).

Symbol	Meaning
w_0	on-demand cost rate per unit capacity per period
w_T	reserved cost rate, $w_T < w_0$ (commitment of length T)
γ	reserved/on-demand ratio $w_T/w_0 \in (0, 1)$
d_0, d_1	base / surge demand per period, d_1 dominates d_0
F_0, F_1	CDFs of base and surge demand
t_d, t_s	surge duration / inter-surge duration
K_0	base capacity (reserved indefinitely)
K_1	total capacity during a surge; $K_1 - K_0$ supplementary
$(x)^+$	$\max(x, 0)$, the on-demand spillover operator
α	fraction of reserved value recovered on cancellation

Cost structure. A firm serves a state-modulated demand: a stationary base state (demand d_0 , fraction $\frac{t_s}{t_s+t_d}$ of time) and a surge state (demand d_1 , fraction $\frac{t_d}{t_s+t_d}$). Holding base capacity K_0 and surge capacity K_1 , the long-run average cost rate is reserved cost plus on-demand spillover priced at w_0 :

$$\begin{aligned} AC(K_0, K_1) = & \underbrace{w_T K_0}_{\text{base contracts}} + \underbrace{w_T (K_1 - K_0) \frac{t_d}{t_s+t_d}}_{\text{supplementary contracts}} \\ & + w_0 \left[\frac{t_s}{t_s+t_d} \mathbb{E}(d_0 - K_0)^+ + \frac{t_d}{t_s+t_d} \mathbb{E}(d_1 - K_1)^+ \right]. \end{aligned} \quad (1)$$

The first two terms are committed reserved spend; the bracket is the expected *on-demand spillover*, the demand that exceeds reserved capacity.

Newsvendor optimality. Minimizing (1) yields a critical-fractile condition. In the single-capacity case ($K_0 = K_1 = K$) the optimum K^* solves [4, Prop. 1]

$$F_0(K^*) + \frac{t_d}{t_s} F_1(K^*) = \frac{w_0 - w_T}{w_0} \left(1 + \frac{t_d}{t_s} \right), \quad (2)$$

and in the dual-capacity case the optimal base and surge capacities are newsvendor quantiles of F_0 and F_1 ,

$$K_0^* = F_0^{-1}(q_0), \quad K_1^* = F_1^{-1}(q_1), \quad (3)$$

with fractiles q_0, q_1 that depend on w_T/w_0 , the cancellation recovery α , and the duration ratio [4, Thm. 1]. Managing the supplementary contracts over time is a two-threshold (s, S) policy on the elapsed cycle time: cancel below a lower threshold, hold to expiration above an upper threshold, and otherwise renew.

Remark 1 (Service level reading). *Ignoring the surge term, (2) reduces to the textbook newsvendor $F(K^*) = 1 - w_T/w_0 = 1 - \gamma$: the optimal reserved capacity covers the $(1 - \gamma)$ quantile of demand. Cheaper reserved capacity (small γ) means reserve more; reserved capacity that is barely discounted (large γ) means reserve little and let demand spill to on-demand.*

From policy to an empirical fractile. Our system does not know F_0, F_1 analytically; it estimates them from telemetry.

For a resource i with observed per-period utilization u_i , the deployed right-sizing rule sizes the required capacity to

$$r_i = q_{0.95}(u_i) \cdot V_i \cdot h, \quad h = 1.3, \quad (4)$$

where $q_{0.95}(u_i)$ is the 95th percentile of utilization, V_i the current resource size, and h a headroom factor. Equation (4) is exactly a *critical-fractile* capacity choice with service level $q \approx 0.95$. Reading it through the remark, a 0.95 service level is cost-optimal only when $\gamma \approx 0.05$ (near-free reserved capacity); for realistic reserved discounts ($\gamma \in [0.6, 0.9]$) the cost-optimal fractile is 0.10–0.40. The deployed heuristic is therefore deliberately *conservative*: it favors avoiding performance-impacting under-provisioning (spillover) over extracting the last dollar, and the $1.3\times$ headroom adds further margin. We make this gap explicit rather than claim the heuristic is cost-optimal; closing it (tuning q to the measured γ) is future work.

IV. SYSTEM: FORECASTING + MILP + RULES

Fig. 1 shows the pipeline. A collection layer gathers inventory, CloudWatch metrics, and Cost Explorer data for ten AWS services (EC2/EBS, RDS, S3, Lambda, DynamoDB, ElastiCache, ECS, ELB, NAT) into a per-account SQLite store; three engines then consume it.

A. Forecasting the demand distribution

The forecasting engine implements five models behind one interface emitting $[date, forecast, lower, upper]$: a naive last-value baseline, a seasonal-naive (weekly) baseline, additive Holt–Winters exponential smoothing (ETS) [26], [27], automatic seasonal ARIMA (SARIMAX) [28], and Prophet [29]. Models are compared by *walk-forward cross-validation*: an expanding training window of at least 120 days, a step of 30 days, and a forecast horizon h ; mean absolute percentage error (MAPE) is averaged over folds. The selected model provides the empirical base distribution $\hat{F}_0$ and prediction intervals; points exceeding $\mu + 2\sigma$ are flagged as surges (the realization of the state-1 / d_1 regime), in the spirit of streaming anomaly detection [31].

B. Sizing capacity with a MILP

For compute (EC2, RDS) the optimizer solves a mixed-integer program with CBC via PuLP [33]. Let $\mathcal{I}$ be the in-scope instances, $\mathcal{C}$ the catalog of candidate types with price p_c , vCPUs v_c , and memory m_c , and $x_{i,c} \in \{0, 1\}$ assign type c to instance i :

$$\begin{aligned} \min_x \quad & \sum_{i \in \mathcal{I}} \sum_{c \in \mathcal{C}} p_c x_{i,c} \\ \text{s.t.} \quad & \sum_c x_{i,c} = 1 & \forall i \in \mathcal{I}, \\ & \sum_c v_c x_{i,c} \geq r_i^{\text{cpu}} & \forall i, \\ & \sum_c m_c x_{i,c} \geq r_i^{\text{mem}} & \forall i, \\ & \sum_i \sum_c p_c x_{i,c} \leq B, \quad x_{i,c} \in \{0, 1\}, \end{aligned} \quad (5)$$

where the capacity requirements $r_i^{\text{cpu}}, r_i^{\text{mem}}$ are the empirical fractiles of (4) and B is a budget cap. The objective picks the cheapest catalog type whose vCPU and memory cover

Algorithm 1 Reservation-policy realization for one account

Require: account telemetry $\mathcal{D}$, catalog $\mathcal{C}$, budget B

```

1:  $\hat{F} \leftarrow \text{SELECTMODEL}(\mathcal{D})$   $\triangleright$  walk-forward CV
2: flag surges where cost  $> \mu + 2\sigma$   $\triangleright$  state-1 demand
3:  $R \leftarrow \emptyset$ 
4: for each compute service  $s \in \{\text{EC2, RDS}\}$  do
5:    $r_i \leftarrow q_{0.95}(u_i) V_i h$  for  $i \in \mathcal{I}_s$   $\triangleright$  Eq. (4)
6:    $x^* \leftarrow \text{solve MILP (5) with } (r_i, B)$ 
7:   add right-size recs where  $x_i^* \neq \text{current}$  and cheaper
8: end for
9: for each rule  $\rho$  in the eight actions do
10:   $R \leftarrow R \cup \rho(\mathcal{D})$   $\triangleright$  reserved switch / waste removal
11: end for
12: de-duplicate  $R$  by (resource, action); keep max saving
13: return  $R, \sum_{r \in R} \Delta_r$ 

```

the fractile-sized load: this is the computational analogue of choosing K_0 in (3), with the residual tail $(d - r_i)^+$ left to on-demand spillover.

C. Choosing reserved vs. on-demand, and removing waste

Eight rule-based actions complete the policy (Algorithm 1). Two *pricing* rules realize the base-contract decision: an on-demand resource with sufficient usage history is switched to a one-year reservation when the reserved rate beats on-demand—i.e., committing the always-on base capacity while leaving variable load on-demand. The remaining six target service-level waste: EBS gp2→gp3 migration, deletion of unattached/idle volumes and idle load balancers, NAT-gateway→VPC-endpoint replacement, S3 intelligent-tiering, DynamoDB capacity-mode switching, and Lambda memory right-sizing. Each action emits a per-resource saving $\Delta_i = c_i^{\text{cur}} - c_i^{\text{new}}$; the engine de-duplicates by (resource, action) and persists the recommendations.

A cold-start advisor (a large-language-model prompt over a nine-question requirements form) covers accounts with no usage history; it is orthogonal to the reservation policy and we do not evaluate its dollar impact here.

V. NUMERICAL STUDY

A. Setup

Following the structure of [4, Sec. 7], we first fix the workload, then evaluate forecasting and optimization. The workload is a single representative account observed for 365 days (2024-07-01 to 2025-06-30), spanning 6 billed service categories and 42 resources (including 8 EC2 instances). Daily cost averages \$74.64 (standard deviation \$22.87, range \$36.10–\$232.52), totaling \$27,243.31 over the year. Defining a *demand surge* as a day exceeding $\mu + 2\sigma = \$120.38$ isolates 5 pronounced spikes (Fig. 2); the largest reaches $z = 6.9\sigma$ above the mean. These are precisely the intermittent state-1 events of the model, and they recur irregularly across the year—exactly the regime the reservation policy targets. EC2 dominates the bill (Fig. 6), so compute right-sizing and reservation carry the largest savings potential.

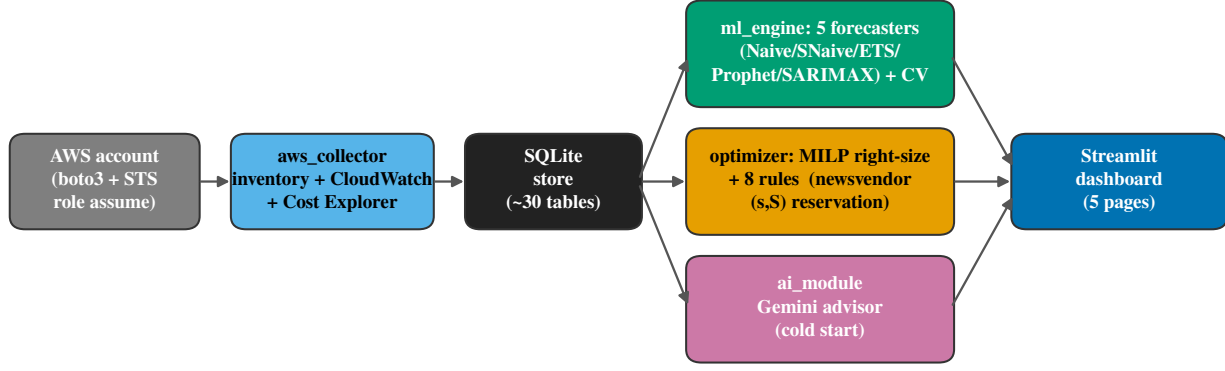


Fig. 1. System architecture. Telemetry is collected per AWS service, stored in SQLite, and consumed by three engines—forecasting, the reservation optimizer (MILP + rules), and an LLM advisor—and surfaced in a dashboard. The optimizer realizes the newsvendor/(s, S) reservation policy of Section III.

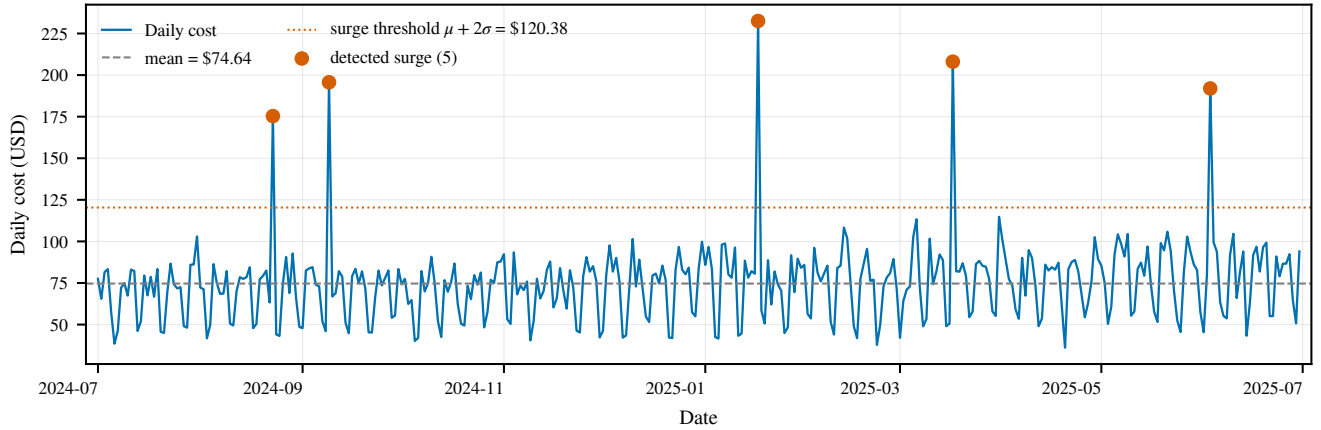


Fig. 2. Daily cost over 365 days. The dotted line is the surge threshold $\mu + 2\sigma = \$120.38$; 5 days (markers) exceed it. Base demand is stationary around \$74.64/day; surges are intermittent and large.

B. Forecasting performance

Table II and Fig. 3 report walk-forward MAPE by model and horizon. Exponential smoothing (ETS) is the most robust model and the best at the deployment-relevant 30-day horizon (11.2%), with the seasonal-naïve baseline competitive at short horizons. Prophet (20.9% at 30 days) trails ETS here—its yearly-seasonality components overfit a single year of data—and the naïve baseline (27.3%) confirms the series is non-trivial. We therefore select ETS. Fig. 4 shows its 30-day holdout forecast with a 95% interval: ETS tracks the base load at 12.0% MAPE with 97% empirical interval coverage, but the lone in-window surge (2025-06-03) pierces the upper band. This is the model’s—and the policy’s—core message: *surges are not forecastable from the base process*; they are the spillover the reservation policy intentionally routes to on-demand rather than provisioning reserved capacity against.

C. Optimization savings

The optimizer produces 19 recommendations (plus 10 advisory items) totaling \$590.40/month. Against the

TABLE II
WALK-FORWARD MAPE (%) BY MODEL AND HORIZON (DAYS). LOWER IS BETTER; BEST PER COLUMN IN BOLD. EXPANDING WINDOW, INITIAL 120 DAYS, STEP 30 DAYS.

Model	$h=7$	$h=14$	$h=30$	$h=60$	$h=90$
ETS (selected)	11.28	11.24	11.20	11.39	12.15
SeasonalNaive	10.18	10.84	12.28	12.66	12.91
Prophet	14.41	20.47	20.93	19.70	18.08
Naive	25.16	27.90	27.26	27.87	27.38

\$2189.48/month bill this is a 27.0% reduction, taking monthly spend to \$1599.08; on the \$1074.83 of resources actually flagged, the reduction is 54.9%. Fig. 5 breaks the savings down by action. The two compute levers dominate: MILP right-sizing of over-provisioned databases (\$362.81, 2 recs) and the reserved/pricing switch on always-on instances (\$208.42, 6 recs) together account for \$571.23, i.e. 96.8% of the total. The six waste-removal rules (delete-unused, NAT,

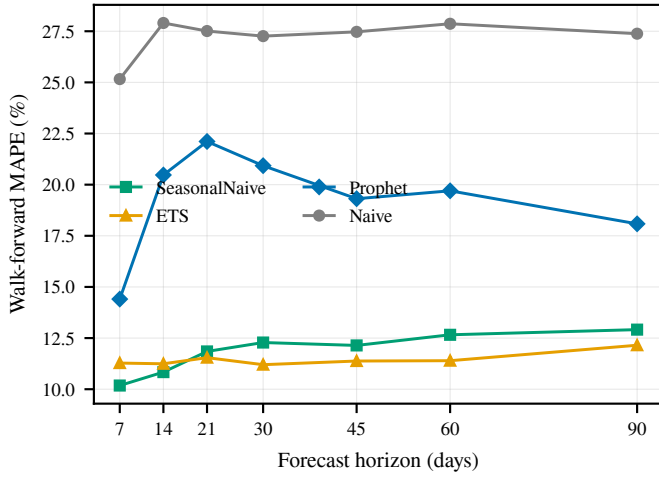


Fig. 3. Walk-forward MAPE vs. horizon. ETS is most accurate and stable; both beat the naive baseline. SARIMAX is excluded for runtime (minutes per fold).

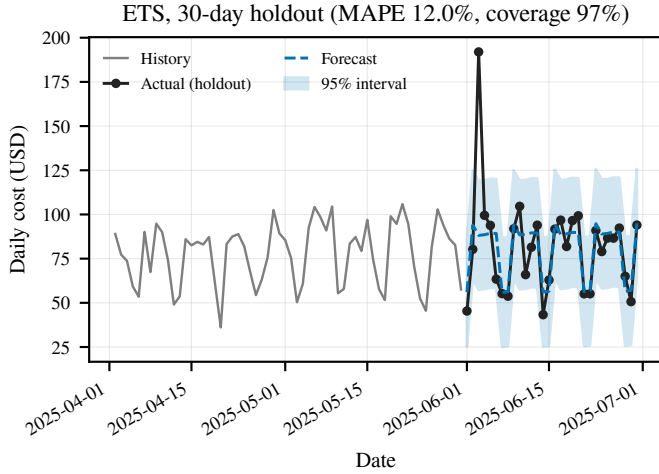


Fig. 4. ETS 30-day holdout forecast with 95% interval. The base load is tracked at 12.0% MAPE; the single surge on 2025-06-03 exceeds the band (on-demand spillover).

EBS, S3, Lambda) contribute the long tail. This distribution matches the model’s intuition: the largest savings come from *committing base capacity correctly* (reservation and right-sizing), not from micro-optimizations. By confidence, 8 high-confidence recommendations carry \$370.52 and 9 medium-confidence ones \$214.21.

D. Policy comparison and sensitivity

The headline comparison mirrors the reference’s cancellation-vs-trade study but on realized cost: the unoptimized (largely on-demand) baseline of \$2189.48/month versus \$1599.08/month under the forecast-driven reservation/right-sizing policy—a 27.0% saving without any change to the workload. The sensitivity that matters for the policy is the service level. As Section III shows, the cost-optimal reserved fractile is $1 - \gamma$; Table III contrasts it

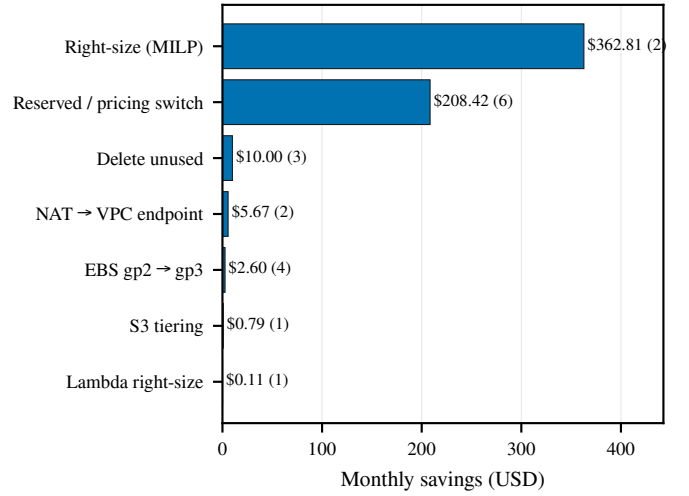


Fig. 5. Recommended monthly savings by optimization action (USD; recommendation count in parentheses). Right-sizing and the reserved/pricing switch dominate.

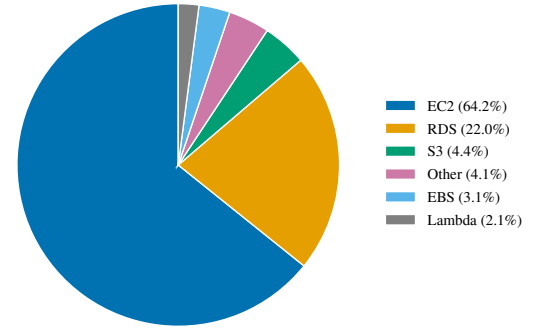


Fig. 6. Share of the monthly bill by service category (Cost Explorer buckets; non-itemized services fold into “Other”). EC2 dominates, so compute reservation carries the largest savings.

with the deployed $q = 0.95$ heuristic across reserved-discount levels analogous to the $\gamma \in \{0.85, 0.90, 0.95\}$ sweep of [4]. The deployed system is uniformly more conservative, which protects against spillover during surges at the cost of some reserved over-commitment—an explicit, tunable trade-off rather than an oversight.

VI. THREATS TO VALIDITY

Workload provenance. The evaluation uses a synthetic, single-account dataset that ships with the system; its base/surge structure and per-service utilization are modeled on publicly documented cloud traces—the Bitbrains GWA-T-12 business-critical workload [3], [34] and the Numenta Anomaly Benchmark [31], [32]—but it is *not* a live AWS account and no trace-conversion generator is bundled with the artifact. Absolute dollar figures are therefore illustrative of the method, not a measured saving on production infrastructure; the *methodology* (Section V) is what transfers.

TABLE III

COST-OPTIMAL RESERVED SERVICE LEVEL $1 - \gamma$ VS. THE DEPLOYED $q=0.95$ HEURISTIC, BY RESERVED/ON-DEMAND RATIO $\gamma = w_T/w_0$.

$\gamma = w_T/w_0$	0.60	0.70	0.85	0.90
optimal fractile $1 - \gamma$	0.40	0.30	0.15	0.10
deployed fractile q	0.95	0.95	0.95	0.95

Policy gap. The deployed system realizes the *static* newsvendor sizing, not the full (s, S) dynamics of [4]: the reserved switch is a per-resource, all-or-nothing one-year commitment rather than a fractional base/supplementary split with cancellation and renewal. Right-sizing (on-demand-priced) and reservation are computed independently and can co-apply to one resource, so the summed saving is an upper estimate of what is jointly realizable. Implementing the two-threshold cancellation/renewal policy is the natural next step.

Forecasting and detection. MAPE excludes zero-cost days and is sensitive to the surge days; the $\mu + 2\sigma$ surge rule is a global threshold rather than the rolling detector used elsewhere in the codebase, and SARIMAX is omitted from the comparison for runtime. The prediction-interval semantics differ across models (ETS uses $\pm 1.96\sigma$ of residuals).

Generality. Results are from one account and one year; multi-account and multi-year validation, and a live-account deployment, remain open.

VII. CONCLUSION

We turned the capacity-reservation policy of Chen, Lei, and Moinzadeh [4]—base and supplementary reserved contracts, on-demand spillover, a newsvendor critical fractile, and a two-threshold (s, S) rule—into a deployable, data-driven system. Forecasting estimates the demand distribution, a mixed-integer program sizes reserved capacity at an empirical critical fractile, and eight rules choose reserved-vs-on-demand contracts and remove service-level waste. On a 365-day multi-service workload, exponential smoothing forecasts the base load at 11.2% MAPE while surges escape the interval as designed, and the optimizer recovers \$590.40/month (27.0% of the bill), concentrated—as the theory predicts—in correctly committing base capacity. We were explicit about the gap to the optimal (s, S) policy and about the synthetic workload. Closing that gap by implementing the cancellation/renewal dynamics, tuning the service level q to the measured reserved discount γ , and validating on live multi-account data are our next steps.

REFERENCES

- [1] Flexera, “State of the cloud report,” Technical report, Flexera Inc., 2024.
- [2] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, “Above the clouds: A Berkeley view of cloud computing,” EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2009-28, 2009.
- [3] S. Shen, V. van Beek, and A. Iosup, “Statistical characterization of business-critical workloads hosted in cloud datacenters,” in *2015 15th IEEE/ACM Int. Symp. on Cluster, Cloud and Grid Computing (CCGrid)*, 2015, pp. 465–474.
- [4] S. Chen, J. Lei, and K. Moinzadeh, “Cost optimization in cloud computing: Capacity reservation for intermittent random demand surges,” *Production and Operations Management*, vol. 33, no. 6, 2024, sSRN working paper 3784812.
- [5] S. Chaisiri, B.-S. Lee, and D. Niyato, “Optimization of resource provisioning cost in cloud computing,” *IEEE Transactions on Services Computing*, vol. 5, no. 2, pp. 164–177, 2012.
- [6] W. Wang, B. Liang, and B. Li, “Optimal online multi-instance acquisition in IaaS clouds,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 12, pp. 3407–3419, 2015.
- [7] L. Hosseini, S. Tang, V. Mookerjee, and C. Sriskandarajah, “A switch in time saves the dime: A model to reduce rental cost in cloud computing,” *Information Systems Research*, vol. 31, no. 3, pp. 753–775, 2020.
- [8] D. a. F. Kirchoff, V. Meyer, R. N. Calheiros, and C. A. F. De Rose, “Evaluating machine learning prediction techniques and their impact on proactive resource provisioning for cloud environments,” *The Journal of Supercomputing*, vol. 80, no. 15, pp. 21 920–21 951, 2024.
- [9] F. Agulló, A. Gutierrez-Torre, J. Torres, and J. L. Berral, “Enhancing the output of time series forecasting algorithms for cloud resource provisioning,” *Future Generation Computer Systems*, vol. 170, p. 107833, 2025.
- [10] G. D. Eppen and A. V. Iyer, “Backup agreements in fashion buying—the value of upstream flexibility,” *Management Science*, vol. 43, no. 11, pp. 1469–1484, 1997.
- [11] M. Erkoç and S. D. Wu, “Managing high-tech capacity expansion via reservation contracts,” *Production and Operations Management*, vol. 14, no. 2, pp. 232–251, 2005.
- [12] L. Huang, J.-S. Song, and J. Tong, “Supply chain planning for random demand surges: Reactive capacity and safety stock,” *Manufacturing & Service Operations Management*, vol. 18, no. 4, pp. 509–524, 2016.
- [13] J.-S. Song and P. Zipkin, “Inventory control in a fluctuating demand environment,” *Operations Research*, vol. 41, no. 2, pp. 351–370, 1993.
- [14] W. Fischer and K. Meier-Hellstern, “The Markov-modulated Poisson process (MMPP) cookbook,” *Performance Evaluation*, vol. 18, no. 2, pp. 149–171, 1993.
- [15] H. Heffes and D. M. Lucantoni, “A Markov modulated characterization of packetized voice and data traffic and related statistical multiplexer performance,” *IEEE Journal on Selected Areas in Communications*, vol. 4, no. 6, pp. 856–868, 1986.
- [16] K. J. Arrow, T. Harris, and J. Marschak, “Optimal inventory policy,” *Econometrica*, vol. 19, no. 3, pp. 250–272, 1951.
- [17] E. Furman and A. Diamant, “Optimal capacity planning for cloud service providers with periodic, time-varying demand,” *European Journal of Operational Research*, vol. 322, no. 1, pp. 133–146, 2025.
- [18] W. Wang, D. Niu, B. Liang, and B. Li, “Dynamic cloud instance acquisition via IaaS cloud brokerage,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 6, pp. 1580–1593, 2015.
- [19] M. C. Cohen, P. W. Keller, V. Mirrokni, and M. Zadimoghaddam, “Over-commitment in cloud services: Bin packing with chance constraints,” *Management Science*, vol. 65, no. 7, pp. 3255–3271, 2019.
- [20] M. A. Núñez, X. Bai, and L. Du, “Leveraging slack capacity in IaaS contract cloud services,” *Production and Operations Management*, vol. 30, no. 4, pp. 883–901, 2021.
- [21] M. E. Arbabian, S. Chen, and K. Moinzadeh, “Capacity expansions with bundled supplies of attributes: An application to server procurement in cloud computing,” *Manufacturing & Service Operations Management*, vol. 23, no. 1, pp. 191–209, 2021.
- [22] S. Chen, K. Moinzadeh, and Y. Tan, “Discount schemes for the preemptible service of a cloud platform with unutilized capacity,” *Information Systems Research*, vol. 32, no. 3, pp. 967–986, 2021.
- [23] S. Chen, H. Lee, and K. Moinzadeh, “Pricing schemes in cloud computing: Utilization-based versus reservation-based,” *Production and Operations Management*, vol. 28, no. 1, pp. 82–102, 2019.
- [24] P. Ambati, D. Irwin, and P. Shenoy, “No reservations: A first look at Amazon’s reserved instance marketplace,” in *12th USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 20)*, 2020.
- [25] I. Pintye, J. Kovács, and R. Lovas, “Enhancing machine learning-based autoscaling for cloud resource orchestration,” *Journal of Grid Computing*, vol. 22, no. 4, p. 68, 2024.
- [26] P. R. Winters, “Forecasting sales by exponentially weighted moving averages,” *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.
- [27] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. OTexts, Melbourne, Australia, 2021, <https://otexts.com/fpp3/>.

- [28] R. J. Hyndman and Y. Khandakar, "Automatic time series forecasting: The forecast package for R," *Journal of Statistical Software*, vol. 27, no. 3, pp. 1–22, 2008.
- [29] S. J. Taylor and B. Letham, "Forecasting at scale," *The American Statistician*, vol. 72, no. 1, pp. 37–45, 2018.
- [30] S. Seabold and J. Perktold, "statsmodels: Econometric and statistical modeling with Python," in *Proc. 9th Python in Science Conference (SciPy 2010)*, 2010, pp. 92–96.
- [31] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, 2017.
- [32] A. Lavin and S. Ahmad, "Evaluating real-time anomaly detection algorithms—the Numenta anomaly benchmark," in *2015 IEEE 14th Int. Conf. on Machine Learning and Applications (ICMLA)*, 2015, pp. 38–44.
- [33] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A linear programming toolkit for Python," Optimization Online technical report, The University of Auckland, 2011, <https://optimization-online.org/2011/09/3178/>.
- [34] A. Iosup, H. Li, M. Jan, S. Anoep, C. Dumitrescu, L. Wolters, and D. H. J. Epema, "The Grid workloads archive," *Future Generation Computer Systems*, vol. 24, no. 7, pp. 672–686, 2008.